Journal of King Saud University Computer and Information Sciences (2025) 37:81
https://doi.org/10.1007/s44443-025-00092-5

ORIGINAL PAPER

A dual scheduling framework for task and resource allocation in clouds using deep reinforcement learning

Jiahui Pan[1] · Yi Wei[1] · Lei Meng[2] · Xiangxu Meng[2]

Received: 21 March 2025 / Accepted: 31 May 2025 / Published online: 23 June 2025
© The Author(s) 2025

Abstract
In the three-tier cloud computing marketplace, Software-as-a-Service (SaaS) providers play a dual role. As consumers of Infrastructure-as-a-Service (IaaS) providers, they rent virtual machine (VM) instances from public cloud platforms on a pay-as-you-go basis to deploy their applications. Meanwhile, as service providers for end users, they are responsible for handling changing workloads consisting of user-submitted tasks with some QoS requirements. Under this business model, how to minimize the cost of leasing VM instances while guaranteeing the quality of service for SaaS applications is a challenging issue. In this paper, we present an intelligent dual scheduling framework for SaaS providers. The QoS-aware task allocation scheduler and the workload-aware VM auto-scaling scheduler work together to achieve efficient task scheduling and elastic resource provisioning. Accordingly, we propose a dual scheduling approach based on deep reinforcement learning, which consists of two layers of Deep Q-Network algorithms to realize these functions. We conducted experiments using simulated workloads and real-world workloads under fixed and elastic resource pool conditions, respectively. Experimental results show that our approach is effective in ensuring higher task success rates, improving resource utilization, shortening average task response time, and significantly reducing the cost of renting VM instances compared to other baseline solutions.

Keywords Cloud computing · Task scheduling · VM Auto-scaling · Deep reinforcement learning

1 Introduction

Cloud computing is an emerging computing paradigm that utilizes technologies such as virtualization to provide users with various computing resources on-demand via the Internet (Rashid and Chaturvedi 2019). Users can easily access and use these resources on a pay-as-you-go basis without purchasing and maintaining their hardware or software. Due to the advantages of reliability, flexibility, and elasticity, more and more companies and organizations have migrated their applications to the cloud. They can rent the appropriate cloud resources from IaaS providers, such as Amazon EC2, to build their software application services at a lower cost (Lee 2021).

For these SaaS providers, they serve a dual function in the cloud marketplace (Mohammed and Zeebaree 2021). On the one hand, as service providers to end users, their SaaS applications must address service requests and perform user tasks to fulfill diverse service requirements. On the other hand, as users of IaaS providers, they pay a fee to lease VM instances from cloud platforms to deploy their SaaS applications. In this business model, SaaS providers aim to ensure the quality of their SaaS application services while minimizing the rental costs of VM instances. Therefore, effectively choosing VM rental plans and allocating user-submitted tasks to suitable resources present significant challenges for SaaS providers.

The workload of SaaS applications can primarily be categorized into two types: long-running web services and batch-processing jobs (Wang et al. 2020). Web services are long-running services, such as business transactions and web search. They are characterized by frequent changes in workload over time and are mainly used to handle short-lived

✉ Yi Wei
weiyi@sdnu.edu.cn

Jiahui Pan
2022317111@stu.sdnu.educn

Lei Meng
lmeng@sdu.edu.cn

Xiangxu Meng
mxx@sdu.edu.cn

1 School of Information Science and Engineering, Shandong Normal University, Jinan 250358, Shandong Province, China

2 School of Software, Shandong University, Jinan 250101, Shandong Province, China

latency-sensitive user requests. Since user-submitted tasks arrive independently and continuously, it is crucial for the task scheduler to efficiently assign tasks to VM instances in time and meet the desired QoS standards. Ensuring an acceptable average response time is critical for such transactional applications. In contrast, batch-processing jobs, such as parallel image rendering and financial analysis, typically involve multiple tasks with complex calculations and processes. These bag-of-task or workflow applications receive user jobs in batches, allowing for offline scheduling due to the absence of a real-time response requirement. The goal of scheduling such batch jobs is generally to assign tasks to available resources to ensure all tasks are completed as soon as possible or minimize the total cost within the expected deadline. Considering that many existing task scheduling methods are tailored for offline rather than real-time scenarios (Arunarani et al. 2019), this paper focuses on online task scheduling for transactional applications in cloud environments.

Furthermore, because the workload of transactional applications changes dynamically over time, their demand for cloud resources varies accordingly. When the resources currently rented by the SaaS provider are less than needed to handle the application workload, it will lead to under-provisioning of resources. This leads to a decline in application performance, thereby impacting service quality. On the contrary, when the SaaS provider uses more resources than needed for the application workload, this results in over-provisioning of resources, causing unnecessary costs. Due to the elastic nature of cloud computing, the resources used by a SaaS application are allowed to be flexibly scaled up or down. Therefore, if the SaaS provider can dynamically adjust resource usage according to changes in application workloads, ensuring the rented VM instances closely match the users' actual needs, it can maintain service quality while reducing costs.

From the perspective of SaaS providers, efficient online task scheduling and application auto-scaling algorithms can improve the performance of SaaS applications, user satisfaction, and the profitability of SaaS providers. However, most existing research (Ibrahim et al. 2021; Singh et al. 2019) has focused on only one of the above aspects and has not considered both comprehensively. For task scheduling, traditional scheduling algorithms are not dynamic enough to adapt to real-time changes in resource states. Meanwhile, it is difficult to balance conflicting objectives such as delay, energy consumption, and cost. Metaheuristic algorithms have high computational complexity and are prone to dimensional catastrophe in large-scale scheduling. Besides, they can suffer from the problem of falling into local optimal solutions. In resource allocation, static allocation refers to a predefined pool of fixed-size resources. This model results in two extremes: when the business volume falls below the preset threshold, a significant portion of resources such as CPU and memory remain idle; whereas, when unexpected traffic exceeds the preset capacity, the system can only manage the overload through queuing or denial of service, leading to service degradation. While traditional elastic allocation incorporates automated scaling mechanisms, it still faces limitations in terms of responsiveness and decision-making. A typical delay occurs when monitoring systems take 5 to 15 minutes to detect load changes, and additional time is required for load-balancing redistribution after horizontal scaling. This latency often forces the adoption of over-scaling strategies during subsequent traffic surges, resulting in a sudden increase in resource costs, which can reach up to 10 to 20 times the daily expenses. Thus, in this paper, we aim to propose an intelligent resource management framework using deep reinforcement learning (DRL) to achieve real-time task scheduling and VM auto-scaling simultaneously. The first layer of the dual scheduling framework utilizes the adaptive learning capability of the DRL to make reasonable task-to-VM assignment decisions, thus improving task success rates. Meanwhile, in the second layer, the workload-aware VM auto-scaling scheduler can make wise VM lease adjustment decisions in a self-learning manner. In the dynamic and unpredictable cloud computing environment, our proposed dual scheduling framework can efficiently reduce the cost of renting VM instances while guaranteeing service quality. The following are the main contributions of our work:

(1) A detailed solution to the problem of task scheduling and resource allocation in the dynamic cloud environment is proposed. The corresponding task model, resource model, and scheduling model for are defined.
(2) A dual scheduling framework based on DRL is proposed. According to the different functions of each layer, specific schemes for the action space, state space and reward function of the first and second layers are designed respectively.
(3) We tested the proposed algorithm using simulated workloads and real-world workloads in different situations separately. The experimental results indicate that our dual scheduling framework exhibits good results in the process of task scheduling and resource allocation.

The rest of the paper is organized as follows. Section 2 reviews the related work on task scheduling and application auto-scaling in cloud computing environments. In Section 3, we present the system architecture and specify the resource management problem for SaaS providers. Section 4 introduces the theoretical concepts related to reinforcement learning and details our proposed DRL-based dual scheduling approach. Section 5 evaluates the effectiveness of our suggested algorithm against other comparative algorithms

and analyzes the experimental results. In Section 6, we summarize the paper and provide an outline for future work.

2 Related work

With the rapid growth of the cloud computing market in recent years, resource management in the cloud computing context has emerged as a critical concern (Abid et al. 2020; Khan et al. 2022). Researchers and scholars have conducted numerous research on task scheduling and application auto-scaling.

2.1 Task scheduling

Task scheduling can be classified into two types, independent task scheduling and associated task scheduling, depending on whether there are dependencies between tasks. In addition to some simple and effective traditional task scheduling methods, such as round-robin scheduling algorithm and Max-Min algorithm (Alhaidari and Balharith 2021; Syed 2020), theoretical model analysis (Yang et al. 2020) and various meta-heuristic algorithms (Kakkottakath et al. 2021; Pradhan et al. 2022) are widely used in designing task scheduling algorithms. For example, Nabi et al. (2022) proposed a task scheduling algorithm based on adaptive particle swarm optimization, which reduces the execution time of a task and improves the throughput and utilization of the system by introducing an adaptive weighted inertia policy. Mao et al. (2019) proposed Decima, a general-purpose task scheduler based on reinforcement learning for workflow tasks. Decima utilizes existing monitoring data and historical load information to adaptively learn scheduling policies. It allocates varying resource capacities to tasks, improving overall task scheduling performance in response to the complex dependencies and prioritization relationships inherent in workflow tasks. Kruekaew and Kimpan (2022) proposed an independent task scheduling approach using an algorithm based on a combination of artificial swarming and Q-learning which makes a balance between resource utilization and cost. Although these methods have achieved significant results in solving task scheduling, they still have some limitations. Theoretical analysis methods are more suitable for relatively stable scenarios, and it is difficult to perceive the dynamism and uncertainties existing in the task scheduling process in cloud computing environments. Heuristic algorithms can successfully solve the scheduling problem of workflows and offline tasks, but they cannot provide efficient online task scheduling support for the SaaS providers' transactional applications.

2.2 Elasticity

Elasticity is a crucial characteristic of cloud computing, enabling cloud platforms or SaaS applications to flexibly and dynamically allocate or deallocate resources at any moment in response to fluctuations in user demand (Barnawi et al. 2020; Alharthi et al. 2024). According to the different ways of scaling, elastic resource provisioning can be categorized into vertical scaling and horizontal scaling (Rzadca et al. 2020). Vertical scaling refers to changing the capacity of system resources or services by increasing or decreasing the capabilities of current computing resources. Horizontal scaling achieves resource elasticity by adding or removing VM instances. Since horizontal scaling is more common for SaaS applications, this paper focuses on horizontal resource scaling. Current mainstream techniques for achieving application auto-scaling include rule-based or threshold-based strategies and methods based on prediction mechanisms (Singh et al. 2019; Tournaire et al. 2023). For instance, Kumar and Sharma (2018) proposed a dynamic scheduling algorithm that increases or decreases elastic resources based on the optimal time interval of the previous round. Liu et al. (2020) proposed an elastic resource management method based on the prediction of edge cloud workload. Mao et al. (2016) designed DeepRM, a multi-resource cluster scheduler based on reinforcement learning, which optimizes resources and adjusts them across multiple objectives by assigning different reinforcement rewards to various resource types. Polamarasetti (2024) presents a system that automatically adjusts resources using machine learning, improving factors such as service response time and service level agreements by forecasting workload on distributed servers and determining the required resources. In addition, platforms like Amazon AWS and Google Cloud offer Auto Scaling, which automatically increases or decreases the number of VM instances based on the load on the compute instances. The Auto Scaling feature in Amazon AWS automatically adjusts the number of Amazon EC2 instances based on the load while ensuring high availability and performance. Google Cloud's task scheduler rationally assigns tasks to the most appropriate VM instances for execution based on resource demand and load.

2.3 Reinforcement learning

Reinforcement learning (Ding et al. 2020) is an artificial intelligence technique whose core idea is to learn how to make optimal decisions through the interaction of an agent with its environment, thereby maximizing long-term rewards. Among them, the Q-learning algorithm has a simple structure and is straightforward to implement and understand, while the parameters can be adjusted to adapt to different learn-

ing environments. It also suffers from the problems of state explosion, overestimation, and exploring the weights. When the state space is too large, the storage and updating of the Q-table become infeasible, Meanwhile, it would be inclined to overestimate the Q-value. The Actor-Critic algorithm is able to better adapt to the problems with continuous action space. However, in some cases, the estimation of the Critic network may encounter an overfitting problem, which can lead to inaccurate policy updates of the Actor network. Meanwhile, the use of multiple networks requires the algorithm to consume more computational resources, especially when dealing with large-scale data. The Proximal Policy Optimization algorithm can effectively avoid the problem of training and updating the Q-table by limiting the policy update step size. It can effectively avoid the performance degradation during the training process and make the training process more stable. However, the algorithm needs to deal with multiple components such as policy and value networks, which raises the difficulty of implementing the algorithm.

Reinforcement learning can be broadly classified into value-based and policy-based approaches. In value-based reinforcement learning, the optimal policy is indirectly derived by estimating the value of state-action pairs. This approach is particularly effective in discrete action spaces or scenarios that require fast convergence. However, it is less suited for continuous action spaces and cannot directly handle continuous control tasks. Additionally, value estimation relies on the randomness of environmental interactions, which can lead to unstable training and high variance, complicating convergence. When the state or action space is too large, Q-tables or neural networks become ineffective for accurate modeling. Yan et al. (2022) proposed an approach based on deep Q-learning to address real-time job scheduling, achieving optimization of both energy consumption and quality of service. However, this method does not account for task delay issues, and the deterministic nature of the Q-learning strategy limits its ability to handle the inherent stochasticity of task scheduling and assignment. In contrast, policy-based reinforcement learning directly optimizes the policy function to maximize the cumulative reward through gradient ascent. While this approach is more suitable for continuous action spaces, it requires extensive interaction data and suffers from slow training speeds. Furthermore, it exhibits high variance and training instability, with gradient estimation often exhibiting large fluctuations, which can cause oscillations in policy updates. Moreover, policy-based methods are prone to falling into local optima, with policy parameters potentially converging to suboptimal solutions in complex environments. Cheng et al. (2024) introduced a task scheduling policy optimization algorithm that incorporates a residual convolutional neural network to enhance the Asynchronous Advantage Actor-Critic (A3C) model. This approach effectively reduces task response times and system energy consumption in cloud-based collaborative environments. However, the A3C algorithm is heavily dependent on hardware, and its performance often degrades in scenarios requiring intensive neural network computations. Consequently, resource costs should be considered when utilizing the A3C algorithm. Zou et al. (2025) presented an online task scheduling framework based on PPO and EMA methods, which improves average task response times. Huang et al. (2023) proposed a cluster task scheduling method using the Deep Deterministic Policy Gradient algorithm to shorten the completion time of cluster tasks. Both methods, however, fail to consider user satisfaction and resource consumption. While the PPO algorithm, as a policy-based method, requires substantial amounts of new data for policy updates, its data utilization tends to be inefficient. The DDPG algorithm, designed for continuous actions, struggles to manage mixed discrete-continuous action spaces.

Deep reinforcement learning (Wang et al. 2022) combines the powerful representation capability of deep learning and the adaptive optimization capability of reinforcement learning, which can better solve optimization and sequential decision-making problems in complex environments. As a result, it is gradually being widely used and shows significant potential in various domains, such as gaming, autonomous driving (Ullah et al. 2025) and 6G network (Prajapati et al. 2025). The application of reinforcement learning in the cloud computing is becoming increasingly important, especially when facing dynamic and complex cloud environments. Traditional scheduling methods may not be well adapted to the complexity of cloud environments when facing large-scale, dynamically changing loads. On the other hand, deep reinforcement learning is able to automatically learn the optimal scheduling strategy by interacting with the environment, and especially exhibits strong capabilities in resource allocation, load balancing, and auto-scaling. Yan et al. (2022) used deep reinforcement learning to process real-time jobs in cloud data centers to optimize energy consumption while improving service quality. Garí et al. (2022) proposed a scaling strategy based on Q-learning algorithm to deal with the scientific workflow scheduling problem. Karthiban and Raj (2020) proposed a fair resource allocation model for green computing based on deep reinforcement learning to provide an efficient resource allocation scheme for network users. Belgacem et al. (2022) proposed a novel resource allocation model, IMARM, based on multi-intelligence and reinforcement learning. This model integrates multi-intelligence features with the reinforcement learning process, enhancing resource allocation performance. The proposed solution demonstrates strong performance in terms of energy consumption and fault tolerance. Li et al. (2022) introduced a novel intelligent optimization approach called collective reinforcement learning for intelligent resource allocation, which ensures efficient sharing of distributed resources while preventing

overconsumption. Hegde et al. (2023) proposed COUNSEL, a deep reinforcement learning-based framework designed to address dynamic workload scheduling and manage the allocation of multi-component resources, effectively handling the computational cost challenges of cloud resources. Wang and Yang (2025) proposed a resource scheduling optimization method based on deep reinforcement learning, which enhances task processing efficiency, improves resource utilization, and effectively controls task migration. Although reinforcement learning has been successfully applied to solve either task scheduling (Shyalika et al. 2020; Hou et al. 2024) or application auto-scaling problems (Wei et al. 2019; Garí et al. 2021), no research has combined the two aspects to date.

3 System framework and problem statement

3.1 System framework

We consider a typical three-tier cloud computing marketplace, as shown in Fig. 1, which involves three roles: IaaS providers, SaaS providers, and end users. IaaS providers employ virtualization technology to abstract underlying physical resources, providing users with on-demand rental services of VM instances as the fundamental unit. SaaS providers rent the appropriate type and number of VM instances from IaaS providers according to actual needs to build resource pools and deploy applications to provide convenient services to end users. End users can submit their task requests with QoS requirements anytime and anywhere. From the perspective of SaaS providers, efficiently managing leased resources and assigning tasks to the appropriate VM instances have become their primary considerations.

Figure 1 depicts the dual scheduling framework designed for SaaS applications. The most important components are the VM auto-scaling and task allocation schedulers. They work interdependently and collaboratively to achieve their respective goals. Specifically, the auto-scaling scheduler is workload-aware and is responsible for adjusting VM rental plans based on resource supply and demand. The task allocation scheduler is QoS-aware, and it can allocate user-submitted tasks to the appropriate VM instances in the resource pool. By adopting appropriate scheduling strategies and algorithms, along with real-time monitoring and dynamic adjustments, efficient VM auto-scaling and task scheduling can be achieved, thereby enhancing application performance and resource utilization.

3.2 Problem description and assumptions

In our model, independent tasks submitted by numerous users constitute the workload of the SaaS application. Tasks arrive at the application in real-time. For most transactional SaaS applications, the task workload varies at different times but generally with some regularity over a long period of time, such as a day or a week. Therefore, we assume that in a given short period of time $periodT_t$, the total number of tasks submitted by users is N_t. The long period with some regularity is $periodWT$. The variation of task workloads and the attributes of each task cannot know in advance. The types of tasks submitted by users are divided into two cat-

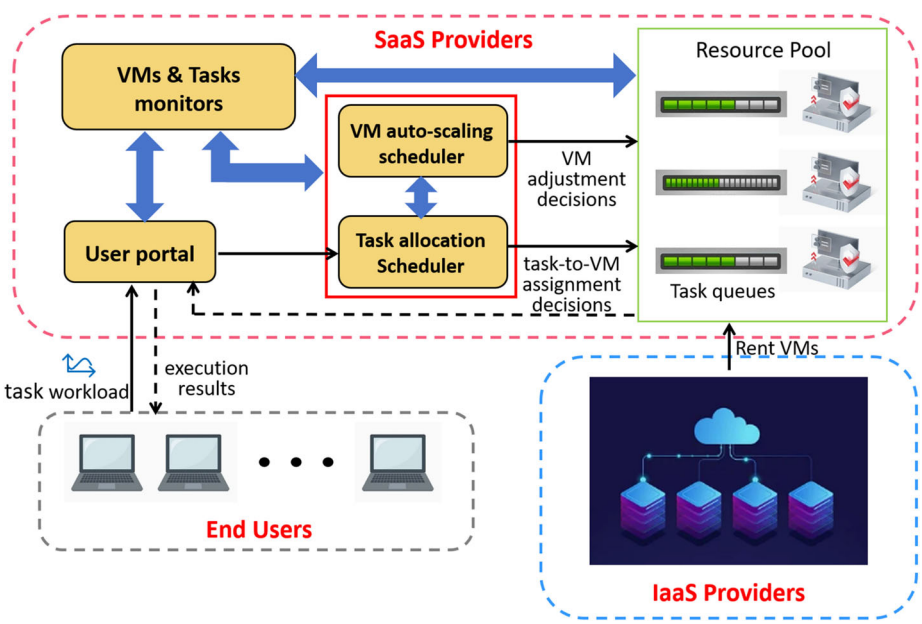

Fig. 1 Dual scheduling framework for SaaS applications

egories: memory-optimized and compute-optimized. Based on the above, each user task can be represented as:

$$task_i = [tID_i, tArrival_i, tType_i, tLen_i, tQoS_i] \tag{1}$$

where tID_i, $tArrival_i$, $tType_i$, and $tLen_i$ represent the serial number, arrival time, type, and length of the task respectively. $tQoS_i$ denotes the QoS requirement for the task.

In the cloud computing market, IaaS providers usually offer various types of VM instances and classify them based on their configurations and computing capacity. The common types include general purpose, graphics processing, compute optimization, and others. Each type of VM instance is optimized for different types of tasks to achieve optimal performance. The speed at which different types of VMs perform the same task can vary significantly. Therefore, a suitable match can reduce task execution time. In our study, it is simplified that there are only two types of VM instances: memory-optimized and compute-optimized. It is assumed that the resource pool contains a limited number of VM instances rented from IaaS providers. With the help of the auto-scaling scheduler in the application module, SaaS providers can flexibly adjust their VM leasing plans based on changes in task workloads. The number of rented VM instances during the period of time $periodT_t$ is M_t. Each VM instance is defined as follows:

$$VM_j = \left[vID_j, vType_j, vPrice_j, vMe_j, vCom_j\right] \tag{2}$$

where vID_j and $vType_j$ represent the serial number and type of the VM instance respectively. $vPrice_j$ represents the rental price of VM_j for $periodT_t$. vMe_j and $vCom_j$ denote the processing speed of VM_j for memory-optimized and compute-optimized tasks respectively.

For the period of time $periodT_t$, the VM rental cost can be calculated as:

$$costT_t = \sum_{i=1}^{M_t} vPrice_j \tag{3}$$

The task allocation scheduler in the application module assigns tasks to specified VM instances based on the scheduling algorithm. We assume that when a new task arrives, it will be promptly processed by the task allocation scheduler and assigned to a specific VM instance. The task will be executed immediately if the assigned VM instance is idle. Conversely, if the designated VM instance is occupied, the task will be placed in the waiting queue for that VM instance. Tasks in the queue are executed on a first-come, first-served basis, ensuring that VM instances execute the tasks in the order of their arrival. This means that the task will remain in the waiting state until all earlier tasks in the queue have been completed. At the same time, the VM instance is limited to executing a single job concurrently, and the task will not be interrupted during processing.

We define the response time $totalT_i$ as the total time that the task $task_i$ stays in the system. It can be computed by the following formula:

$$totalT_i = exeT_i + waitT_i \tag{4}$$

where $exeT_i$ represents the execution time of $task_i$ on the allocated VM instance, and $waitT_i$ represents the waiting time of $task_i$ in the VM waiting queue. Assuming that the task $task_i$ is assigned to VM instance VM_j, the execution time $exeT_{ij}$ can be calculated according to the attributes of the task and the assigned VM instance:

$$exeT_{ij} = \begin{cases} tLen_i/vMe_j, & if\ task_i\ is\ memory-optimized \\ tLen_i/vCom_j, & if\ task_i\ is\ compute-optimized \end{cases} \tag{5}$$

where $tLen_i$ is the length of $task_i$, and vMe_j and $vCom_j$ are the processing speeds of VM instance VM_j for different types of tasks respectively.

The waiting time of $task_i$ on VM instance VM_j is determined by the number of tasks in the waiting queue, so $waitT_{ij}$ can be defined as:

$$waitT_{ij} = \begin{cases} 0, & if\ wqLen_j = 0 \\ idleT_{ij} - tArrival_i, & else \end{cases} \tag{6}$$

where $wqLen_j$ is the number of tasks in the waiting queue of VM instance VM_j at this moment. $idleT_{ij}$ is the idle time of VM_j to execute $task_i$, which can be expressed as:

$$idleT_{ij} = \begin{cases} idleT_{i'j} + exeT_{i'}, & if\ idleT_{i'j} > tArrival_{i'} \\ tArrival_{i'} + exeT_{i'}, & else \end{cases} \tag{7}$$

where $task_{i'}$ is the previous task of $task_i$ in the waiting queue of VM instance VM_j. $idleT_{i'j}$ is the idle time of VM_j to execute $task_{i'}$, $tArrival_{i'}$ and $exeT_{i'}$ are the arrival time and execution time of $task_{i'}$ on VM_j respectively.

In the field of task scheduling in cloud computing, task response time serves as a critical QoS metric for transactional SaaS applications. For a specific task, if its total time in the system is within the user's acceptable response time, the task is judged to be successful; otherwise, the task is judged to be a failure. We use the following formula to describe whether task $task_i$ is successfully executed on VM instance VM_j.

$$success(task_i, VM_j) = \begin{cases} 1, & if\ totalT_i \le tQoS_i \\ 0, & else \end{cases} \tag{8}$$

To maximize the benefits of SaaS providers, our dual scheduling framework aims to increase task success rates while reducing the total rental cost of VM instances. To achieve this goal, a workload-aware auto-scaling scheduler and a QoS-aware task allocation scheduler are necessary. They work together to enable flexible adjustment of the number of rental VM instances and optimal task-to-VM assignments.

4 DRL-based dual scheduling approach

As we mentioned in Section 3.1, task allocation and auto-scaling schedulers are the core components of the dual scheduling framework, determining the cost and quality of service of the SaaS applications. The effectiveness of schedulers is significantly influenced by the underlying algorithms they use. In the dynamic cloud environment, workloads of transactional SaaS applications are constantly changing and unpredictable. Deep reinforcement learning algorithms are particularly well-suited for solving complex sequential decision problems. Therefore, we propose a DRL-based dual scheduling approach for SaaS providers. In this section, we first introduce theoretical knowledge related to reinforcement learning. Next, we detail the specific designs of the task allocation scheduler and VM auto-scaling scheduler, respectively. Finally, the algorithm implementation of the DRL-based dual scheduling approach is presented.

4.1 Theoretical foundation

Reinforcement learning (RL) is a machine learning (Khan et al. 2025, 2024) method that learns which actions to take by engaging with the environment to maximize cumulative rewards (Sutton 2018). Markov Decision Processes (MDPs) can be used to describe the training process of reinforcement learning. At each decision time t, the agent notices the current state s_t and takes an action a_t. After executing the action, the environment transitions to a new state s_{t+1}. Simultaneously, the agent receives a reward r_t calculated by the reward function, which reflects the quality of the decision.

Q-Learning is a model-free RL algorithm based on value iteration, aiming to identify the optimal policy by learning the Q-function through interactions with the environment (Clifton and Laber 2020). The updating formula for the Q-function can be described as:

$$Q_{t+1}(s,a) = Q_t(s,a) + \alpha\left(r + \gamma \max_{a'} Q_t\left(s',a'\right) - Q_t(s,a)\right) \tag{9}$$

where $\alpha \in (0, 1)$ is the learning rate, which finds how much the new information affects the updating of the Q-value. $\gamma \in (0, 1)$ is the discount factor, which measures the significance for future rewards. r is the immediate reward. s' is the next state of s after executing the action a. $\max_{a'} Q_t\left(s',a'\right)$ denotes the Q-value for selecting the optimal action a' in the new state s'.

For Q-Learning, the agent needs to strike a balance between exploration and exploitation. A common exploration strategy is the ϵ-greedy strategy, where actions are randomly chosen with a probability of ϵ and the optimal action is chosen with a probability of $1-\epsilon$.

Deep Reinforcement Learning (DRL) is a technique that combines RL and deep learning (Wang et al. 2022). DRL uses deep neural networks to process high-dimensional state spaces and approximate the Q-function for efficient decision-making and optimization in complex environments. RL agent learns by interacting with the environment to maximize cumulative rewards. Deep Q-Network (DQN) is a groundbreaking algorithm in the field of deep reinforcement learning, which we employ in our proposed approach.

4.2 QoS-aware task allocation scheduler

The QoS-aware task allocation scheduler in our dual scheduling framework focuses on the assignment of tasks, i.e., how to choose the most appropriate VM instance from the resource pool for the current task. The following is the RL model that addresses the online task scheduling problem.

State and action The observation $state_i$ of the task allocation scheduler for the i-th task-to-VM decision records the characteristics of the task and the usage of VM instances in the resource pool. It is denoted as:

$$state_i = [tType_i, waitT_{i1}, waitT_{i2}, \ldots, waitT_{iM}] \tag{10}$$

where M is the number of VM instances in the resources pool at this time, $waitT_{ij}$ stands for the waiting time if the task $task_i$ is assigned to the j-th VM instance.

The action space $action_i$ of the task allocation scheduler for the i-th task-to-VM decision can be recorded as a set of all VM instances in the resource pool, i.e.,

$$action_i = [VM_1, VM_2, VM_3, \ldots, VM_M] \tag{11}$$

Reward For transactional SaaS applications, the task response time is a main metric which determines the application performance and user satisfaction. Because of that, we set the reward function for $task_i$ as follows:

$$reward_i = -totalT_i/exeT_i \tag{12}$$

where $totalT_i$ is the total time that $task_i$ stays on the assigned VM instance and $exeT_i$ is the actual execution time of $task_i$. The fractional form facilitates multivariate trade-offs and balancing. In task scheduling, multiple variables are often considered simultaneously, and direct summation can lead to one variable dominating the reward function due to differences in scale. In contrast, the fractional form maps indicators of varying scales to a uniform scale, preventing one variable from overwhelming others. The negative form ensures the stability of the gradient by significantly decreasing the reward as the independent variable increases. This clear gradient direction accelerates the convergence of the Q-network. Additionally, negative rewards generate large negative values during the early stages of optimization, encouraging the agent to actively explore better actions. Accordingly, a small percentage of the waiting time in the total time indicates a better result for the assignment.

4.3 Workload-aware VM auto-scaling scheduler

The workload-aware VM auto-scaling scheduler in our dual scheduling framework has responsibility for constantly adjusting the amount of rental VM instances according to the task workload variations and the utilization of currently rented resources. The VM rental plan is adjusted at regular intervals. When the number of tasks is low in a certain period, the number of VM instances can be appropriately reduced to avoid wasting resource and to lower rental costs. Conversely, when VM instances in resource pool are saturated, renting more VM instances can ensure the success rate of user tasks and enhance user satisfaction. The RL model for addressing the VM auto-scaling scheduling problem is formulated as follows.

State and action The observation $state_t$ when the VM auto-scaling scheduler makes the t-th VM rental adjustment plan can be expressed as:

$$state_t = [N_{t-1}, M_{t-1}, Time_t, avgT_{t-1}, suc_{t-1}, util_{t-1}] \tag{13}$$

where N_{t-1} and M_{t-1} represent the number of tasks submitted by users and the number of leased VM instances during the last period of time $periodT_{t-1}$ respectively. $Time_t$ is a specific time within the workload period $periodWT$. $avgT_{t-1}$ and suc_{t-1} are the average response time and success rate of all tasks during this period, respectively. $util_{t-1}$ is the resource utilization rate of all VM instances in the resource pool during $periodT_{t-1}$.

In the action selection phase, the agent chooses to increase or decrease the number of rental VM instances according to current state $state_t$. The set of available actions is the range of changes in the number of VM instances, which we denote as:

$$action_t = [-k, 1-k, \ldots, 0, \ldots, k-1, k] \tag{14}$$

where k stands for the maximum range of VM instances that can be added or subtracted. For example, action k indicates that k VM instances will be added, while action $-k$ indicates that k VM instances will be removed.

Reward Adjusting the number of leased VM instances by the VM auto-scaling scheduler affects not only the resource utilization but also the response time of tasks. The optimization goal of the dual scheduling framework is to reduce the cost of renting VM instances while ensuring the quality of service. Therefore, the design of the reward function needs to consider both the success rate of tasks and the utilization rate of VM instances. We define the reward function as follows:

$$reward_t = \ln^{5*util_t+1} + \alpha * P \tag{15}$$

where $util_t$ and suc_t are the resource utilization rate and the task success rate during $periodT_t$. The logarithmic form of the reward function better compresses the reward range and stabilizes training. In resource utilization optimization, when the reward function is linearly related to the utilization rate, the logarithmic compression can better balance the learning weight of the load nodes, while the logarithmic form facilitates better handling of multiplicative relationships, simplifies parameter optimization, and suppresses the interference of outliers. However, the logarithmic form is sensitive to negative values; if the independent variable takes a negative value, it may lead to calculation errors. Since the utilization rate never takes negative values, this problem can be avoided. At the same time, the compression of information may be lost in the process of the subtle differences in the original rewards; we can adjust the coefficients of the utilization rate to solve the problem. P denotes a penalty that needs to be given to the reward when the success rate falls below a certain threshold. α reflects the weight of the penalty to the reward. P is defined as follows:

$$P = \begin{cases} 0 & suc_t \geq 0.9 \\ \exp^{0.9-suc_t} + 0.8 & else \end{cases} \tag{16}$$

The exponential form of the penalty term amplifies the importance of task success, with penalty weights increasing exponentially as task success decreases, thereby forcing the agent to prioritize task success. This form also helps prevent reward densification while limiting the reward range, thus mitigating the risk of gradient explosion. However, the exponential penalty term is prone to numerical instability

and local optima, so to avoid excessively large penalty terms, the success rate is clipped. In a multi-objective reward function, the priority of different objectives is regulated by the weight coefficient α. A higher weight increases the influence of the corresponding objective on the strategy, while an imbalance in the weights may bias the strategy towards a single objective, neglecting other metrics. The weight factor α is manually adjusted based on domain knowledge. First, the priority ranking of objectives is clarified to determine the parameter range. Then, the weights are proportionally assigned in stages according to business requirements and fine-tuned through experimentation. Finally, the behavior of the strategies during training is monitored to correct any parameter imbalance in a timely manner. Initially, the penalty weight α is set to 1, and the penalty term is gradually strengthened by increasing the weight to ensure that both parts of the reward function have comparable magnitudes. After several experimental tests, the penalty weight α is ultimately set to 3 to ensure the overload penalty has a significant effect.

4.4 Algorithm implementation

In the three-tier cloud market, the task workloads of SaaS applications are unpredictable and dynamically changing. Allocating the continuously arriving tasks submitted by users to limited VM instances and adjusting the number of rented VM instances based on resource usage and workload variations are two significant challenges for SaaS providers. SaaS providers can only monitor the status of their rented VM instances in the resource pool and the current task workload, but cannot predict the future arrival times and resource requirements of tasks. Therefore, we need intelligent online scheduling algorithms that can be adaptive to solve such complex problems. Compared to traditional task scheduling, which relies on predefined rules and static models, DRL algorithms can more effectively adapt to dynamic environmental changes without any prior knowledge of the environment.

Based on the designs of the QoS-aware task allocation scheduler and workload-aware VM auto-scaling scheduler, we propose a DRL-based dual scheduling approach for SaaS providers that can significantly improve cloud resource utilization and system performance through continuous learning and improvement. It consists of two layers of DQN algorithms. The first layer DQN algorithm is responsible for task allocation, and the second layer DQN algorithm is responsible for dynamically adjusting the number of leased VM instances. These two DQN layers interact with each other. Each DQN algorithm comprises two cooperative phases: the online decision phase and the offline training phase. The framework diagram of the approach is shown in Fig. 2.

Online decision-making phase It is responsible for making task-to-VM assignment decisions and VM adjustment decisions for the task allocation scheduler and VM auto-scaling scheduler, respectively. Specifically, for the task allocation scheduler, when a new user-submitted task arrives at the system, the DRL agent will observe the present condition of the environment and use DNN to calculate q-values for all rented VM instances at the current state. A VM instance in the resource pool will be assigned to execute this task based on the ϵ-greedy policy. Additionally, the DRL agent will receive an immediate reward for the task scheduling decision. For the VM auto-scaling scheduler, the attributes of the tasks and VM instances over a certain period of time are

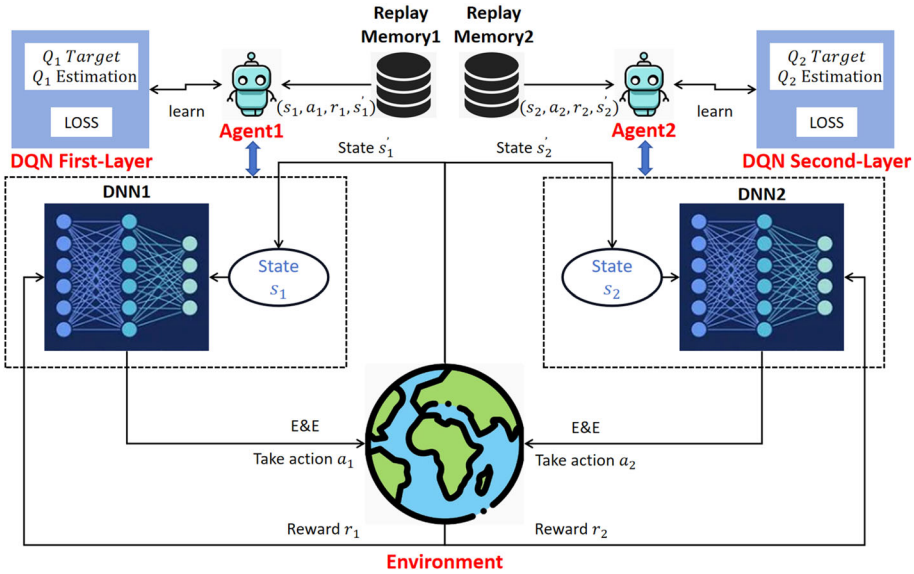

Fig. 2 The dual DQN algorithm framework

recorded and used as part of the state. The DRL agent adjusts the number of VM instances based on the current state and the output values from the DNN.

Offline training phase Deep neural networks (DNNs) (Samek et al. 2021; Meng et al. 2020) are the core components responsible for approximating value functions, policy functions, and model functions in DRL. DNNs deal with high-dimensional and complex state spaces using multilayer neuron structures and nonlinear activation functions. Due to their powerful generalization ability and flexible adaptation, DNNs offer significant advantages in solving our problem. In this paper, we construct two distinct DNNs for the two layers of DQN algorithms, respectively. In the offline training phase, we use past choices and corresponding results to train the underlying DNN to continuously correct and update the state-action value function. Experience replay and fixed target network techniques (Wei et al. 2018) are adopted to improve the stability and performance of the DQN algorithms. Experience replay technique stores past experiences (s, a, r, s') in a replay buffer and randomly sampled to train the agent. This disrupts the link between successive experiences, improving the efficacy and stability of the learning process. Fixed target network technique involves maintaining a separate target network $Q(s', a'; \theta')$, whose parameters θ' are updated less frequently than the main network $Q(s, a; \theta)$. This target network is used to generate target Q-values. This technique can reduce oscillations and divergence during training.

The pseudo code of the DRL-based dual scheduling approach is shown in Algorithm 1.

Algorithm 1 Dual DQN task scheduling framework.

Input: Initialize $\varepsilon_1, \varepsilon_2, \alpha_1, \alpha_2, \gamma_1, \gamma_2$, learning frequency f_1, f_2, start learning time τ_1, τ_2, replay period η_1, η_2
Initialize replay memory D_1, D_2 with capacity N_1, N_2
Initialize action-value function Q_1, Q_2 with random weights θ_1, θ_2
Initialize target action-value function Q'_1, Q'_2 with weights $\theta'_1 = \theta_1, \theta'_2 = \theta_2$

```
for t = 1 to T do
    if a new task arrives then
        Selection of action a_1 based on ε_1-greedy policy
        Execute action a_1, get reward r_1 and next state s'_1
        Store transition (s_1, a_1, r_1, s'_1) in replay memory D_1
        if t > τ_1 and t%f_1 == 0 then
            Sample random minibatch of transitions (s_i, a_i, r_i, s'_i) from D_1
            for each transition (s_i, a_i, r_i, s'_i) do
                Perform the step on ((r_i + γ_1 max_a' Q'_1(s'_i, a'; θ'_1)) − Q_1(s_i, a_i; θ_1))^2
            end for
        end if
        if t%η_1 == 0 then
            Update target network θ'_1 = θ_1
        end if
    end if
    if a fixed time interval periodT is reached then
        Selection of action a_2 based on ε_2-greedy policy
        Execute action a_2, get reward r_2 and next state s'_2
        Store transition (s_2, a_2, r_2, s'_2) in replay memory D_2
        if t > τ_2 and t%f_2 == 0 then
            Sample random minibatch of transitions (s_j, a_j, r_j, s'_j) from D_2
            for each transition (s_j, a_j, r_j, s'_j) do
                Perform the step on ((r_j + γ_2 max_a' Q'_2(s'_j, a'; θ'_2)) − Q_2(s_j, a_j; θ_2))^2
            end for
        end if
        if t%η_2 == 0 then
            Update target network θ'_2 = θ_2
        end if
    end if
end for
```

5 Experiments and results

In this section, we assess the effectiveness of the DRL-based dual scheduling approach through a variety of experiments. First, we describe the experimental setup. Then, by comparing with the baseline algorithms, we validate the performance of a single-layer DQN algorithm designed for the task allocation scheduler and the performance of a combined dual DQN algorithm designed for the task allocation scheduler and the VM auto-scaling scheduler. All experiments were performed in Python 3.11 using TensorFlow on a Windows system with an Intel Core I5-7300HQ and 12GB of RAM .

5.1 Experimental setup

All the experiments in this paper are conducted using a self-designed simulation simulator, which is based on the CloudSim platform. Some of its functionalities, including virtualized resource modeling, task scheduling strategies, and dynamic scenario simulation, are incorporated to enhance the simulator developed for this study. However, since CloudSim lacks a graphical interface, this paper integrates Matplotlib to visualize and analyze the experimental results.

Tasks and VMs The transactional SaaS application provider receives workloads in the form of many continuous memory-optimized and compute-optimized tasks submitted by users. The probability distributions of task arrivals and task types change unpredictably over time. By default, the length of these tasks is normally distributed. We assume that the SaaS provider rents up to 10 VM instances from the IaaS provider on a pay-as-you-go basis. The VM instances are rented for USD 0.125 per minute. These VM instances consist

Table 1 Processing capacity of VM instances

	Memory-optimized tasks	Compute-optimized tasks
Memory-optimized VM	AVG 1200 AVG 800	AVG 600 AVG 400
Compute-optimized VM	AVG 600 AVG 400	AVG 1200 AVG 800

of five memory-optimized and five compute-optimized VM instances, with two different processing capabilities for each VM type. Table 1 lists the average processing power of these VM instances for different types of tasks.

DNN construction and DQN parameters Our proposed DQN-based dual scheduling approach consists of two DNNs which are responsible for processing different features. Each DNN we design consists of one input layer, two hidden layers, and one output layer, each containing 20 neurons. Hyperparameters in DQN algorithms have an important impact on the training process and the final model. Primary hyperparameters include the exploration rate, learning rate, discount factor, batch size, etc. The hyperparameters that significantly influence the experiments are the learning rate, the experience replay buffer size, and the batch size. The learning rate controls the balance between convergence speed and stability; a rate that is too high may cause oscillations, while a rate that is too low leads to inefficiency. The experience replay buffer manages sample diversity and timeliness, and its size must be appropriately matched to the corresponding task, with adjustments based on the frequency and changes in task scheduling relative to the available resources. The batch size affects the quality of gradient estimation, requiring a trade-off between variance and computational overhead. We tried different combinations of parameters and selected the optimal parameter settings for the dual DQN algorithm, as shown in Table 2.

Comparison algorithms To evaluate the performance of our proposed DRL-based scheduler algorithm, we use different comparison algorithms under fixed and elastic resource pool conditions. To validate the performance of the single DQN algorithm, we use the randomized algorithm, polling algorithm, first-come-first-served algorithm, and Tetris algorithm (Arunarani et al. 2019; Psychas and Ghaderi 2018; Kaur et al. 2022; Grandl et al. 2014). Among them, Random algorithm is one of the simplest task allocation methods, where the scheduler randomly selects an available computing resource to execute a task without considering the current load of the resource or the execution time of the task. Polling algorithm is a fair allocation strategy, where the scheduler sequentially assigns the tasks to the available computing resources in a round-robin order. Each resource receives tasks in turn and does not skip a resource unless it is unavailable. The FCFS algorithm is an intuitive scheduling strategy for task scheduling. In a multi-VM environment, when a new task arrives, the algorithm selects the VM instance with the shortest waiting time to execute the task. The Tetris algorithm schedules tasks based on the resource requirements and system resources at the time of task arrival. All VM instances are dynamically sorted based on their current properties. The newly arrived task will be assigned according to the priority of VM instances. For the dual DQN algorithm, we choose to compare it with the single-layer DQN algorithm and two other double-layer algorithms, which will be presented in Section 5.5.

Table 2 Settings of hyperparameters

	First layer DQN	Second layer DQN
Batch size	15	30
Size of replay buffer	400	300
Number of episodes	50	6
Epsilon	0.01	0.01
Learning rate	0.0001	0.001
Discount factor	0.9	0.95

5.2 Time complexity and scalability analysis

Let T denote the total number of tasks, and V represent the number of VM instances available for allocation. For task scheduling analysis, in the random algorithm, a VM is randomly selected for each task per scheduling cycle, with no dependence on the historical state. The time for each selection is O(1), and the total scheduling time for T tasks is O(T). The polling algorithm is similar to the random algorithm in that tasks are assigned sequentially to VM instances in a fixed order, with each selection taking O(1) time, resulting in a total scheduling time of O(T). The FCFS algorithm assigns tasks to idle VM instances, leading to a selection time of O(V) per task, and the total time for scheduling T tasks is O(TV). The Tetris algorithm, like the FCFS algorithm, has a total scheduling time complexity of O(TV). The ARIMA algorithm consists of two phases: model training and numerical prediction. The model training phase is divided into two steps: difference operation and autoregressive and moving average parameter estimation. Assuming the length of the training time series is n, the time complexity for the difference operation is $O(dn^2)$, while the moving average parameter estimation involves a matrix inverse operation with complexity $O((p+q)^3)$. Therefore, the combined time complexity of the training phase is $O(dn^2 + (p+q)^3)$, which is typically simplified to $O(n^2)$ in practice. The prediction phase forecasts

future load changes based on the number of VMs, with a time complexity of O(V(p+q)). Assuming that k tasks are processed every five minutes, the overall time complexity of the ARIMA algorithm is $O(T/k(n^2 + V(p+q)))$.

The DQN algorithm predicts the Q-value for each VM using a neural network and subsequently selects the optimal VM for task allocation. Let L represent the number of neural network layers and h the dimension of the hidden layers. Task scheduling can be divided into three phases: state encoding, neural network inference, and action selection. In the state encoding phase, a state vector is constructed by collecting the state of each VM, with a complexity of O(V). The neural network inference phase involves inputting the state vector into the neural network and outputting the Q-values for each VM, with a complexity of $O(Lh^2)$. In the action selection phase, the VM with the highest Q-value is selected, with a complexity of O(V). Therefore, the total time complexity of the DQN algorithm is $O(T(V + Lh^2))$, where the fixed network structure $O(Lh^2)$ is typically considered constant, allowing the total time complexity to be simplified to O(TV).

In the analysis of resource allocation, this paper assumes that the elasticity scaling of resources is adjusted once every five minutes during task scheduling. The adjustment of resource elasticity scaling is integrated into the task scheduling process, allowing the time complexity of the two-tier algorithms to be determined by the direct accumulation of task scheduling and resource elasticity scaling. For the dual-layer scheduling algorithms, the time complexity of the FCFS-DQN algorithm is O(2TV), the time complexity of the DQN-ARIMA algorithm is $O(TV + T/k(n^2 + V(p+q)))$, and the time complexity of the Dual DQN algorithm is O(2TV).

Traditional scheduling algorithms, while having low time complexity, are not particularly effective in task scheduling. In contrast, metaheuristic and predictive model algorithms can exhibit instability in their performance due to the influence of environmental factors and parameters, which cause variations. The Dual DQN algorithm, however, is primarily influenced by the number of tasks and virtual machines. In practical applications, by appropriately allocating computational resources, it can effectively reduce both time and resource costs.

In large-scale task scheduling scenarios with complex resource pools, the scalability of scheduling frameworks faces significant challenges. The primary challenge is the state space explosion and the linear growth of the action space. Each resource pool's state must be encoded as a state vector, with a state dimension of mV, assuming each resource pool has m-dimensional features. As the resource pool grows, the dimension of the action space increases linearly. To address this, we can compress both the state and action spaces by retaining only key resource metrics, thereby reducing the m-dimensionality. Alternatively, principal component analysis (PCA) can be used to compress the state space, while grouping resource pools by type or region can reduce the action space's dimensionality. Additionally, a pre-filtering mechanism can be implemented to dynamically filter candidate resource pools based on their real-time status, and the Q-value can then be calculated only for the streamlined resource pools. The second challenge is the neural network inference delay and the experience replay overhead. In practice, a distributed and parallelized structure can be adopted to perform computations more efficiently. This involves distributing the neural network parameters across multiple servers, processing state encoding and Q-value computation for different resource pools in parallel, and dividing tasks and resource pools into sub-clusters, where the lightweight Dual DQN algorithm is executed independently for each sub-cluster. Alternatively, the scheduling model can be made more lightweight and accelerated by removing redundant neurons, reducing the pruning rate of the model network, and lowering the number of parameters and computations. The model can also be converted to a low-precision format, such as INT8, to further reduce the computational load.

While the scalability of this scheduling model supports theoretical scenarios, real-world applications require consideration of factors such as real-time business demands, cluster scale, and high-precision sensitivity. Therefore, in practice, it is necessary to employ various strategies to fine-tune the model to adapt to different operational environments.

5.3 Parametric analysis

For the weight α of the penalty term designed in the reward function of the VM Auto-scaling Scheduler, we conducted experiments with different values of weight α in a simulated environment. The convergence of the algorithm is verified by calculating the standard deviation of the hourly reward values, where smaller values indicate more prominent convergence of the rewards. Since the range of standard deviation varies due to the different weights, the experimental results are normalized, and the specific results are shown in Fig. 3. We take values 0, 3, 6, and 9 for the weight α. When α takes 0, it means that the VM Auto-scaling Scheduler only considers the task's utilization requirement and ignores the task's success requirement. In this case it can be found that the reward has been fluctuating and there is no convergence trend. It shows that considering only the utilization of the tasks may lead to a situation where the rewards cannot converge. When α is taken as 6 and 9, respectively, it is found that the rewards tend to converge but cannot stably converge to a fixed range, and the fluctuation range when α is 9 is more significant than that when α is 6. It shows that in both cases the algorithm has convergence but fails to achieve stable convergence. When α is 3, it can be observed that there is a clear changing trend in the convergence of the reward during the initial learning

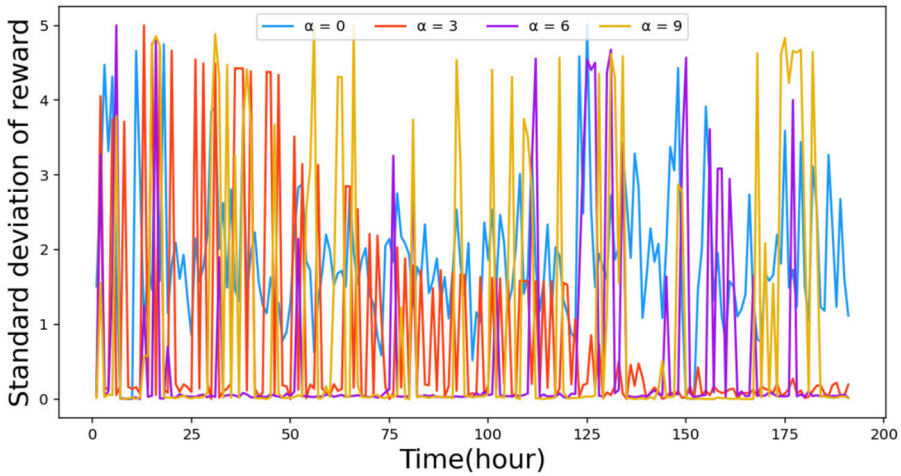

Fig. 3 Convergence performance for different values of parameter α

phase due to the stochastic nature of the environment. As a result, the standard deviation of the reward is relatively large. As the learning progresses, the standard deviation of the reward is gradually decreases and eventually converges to a stable range. This indicates that when α is 3, the reward exhibits a certain degree of convergence and stability.

5.4 Experiments with the single-layer DQN algorithm

In this subsection, we simulate realistic task arrival trends to evaluate the performance of the single-layer DQN algorithm designed for task scheduling under a fixed resource pool. The number of leased VM instances is fixed at 10. We generate datasets by simulating realistic task arrivals conforming to a Poisson distribution. The task arrival rate varies throughout the day. We set the number of tasks arriving per minute between 50 and 500. The ratio of memory-optimized tasks to compute-optimized tasks is 1:1. The total length of the task workloads is 8 days. An example of the task variations in a day is shown in Fig. 4.

We tested our single-layer DQN algorithm in three different cases. In the first case, we categorized the task load into low, medium, and high. We multiply the original task arrival rate by different multipliers to generate different task loads. The number of tasks under different loads is different. The higher the load, the more tasks there are. In the second case, we categorize the task length into short, medium, and long. We modify the task length under the criterion of short-length tasks to produce medium and long tasks. The longer the task length, the longer the task processing time, and the longer the system processing time with no change in the VM instances. In the third case, we vary the maximum number of tasks to simulate a sudden rise in the number of tasks during busy times. We then test our algorithm's ability to handle this scenario. We modify the task load during the peak period of the

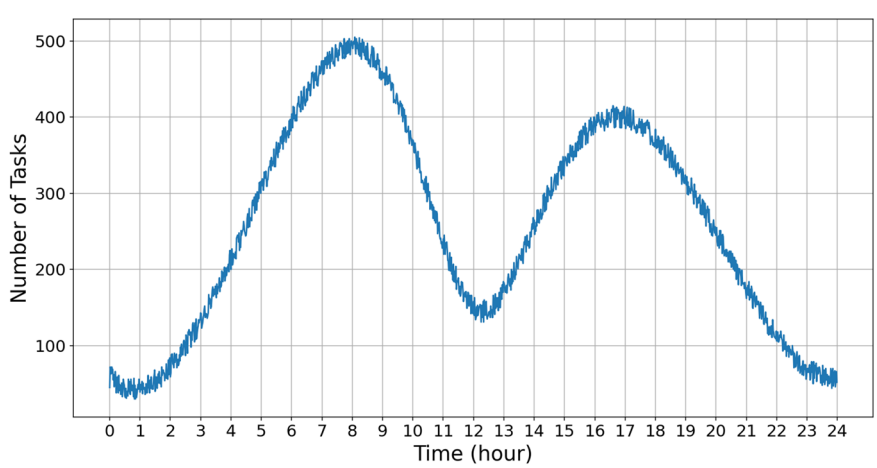

Fig. 4 Task workload variations in a day

Table 3 Comparison results of scheduling algorithms under different task loads

Categorization	Metric	Random	Polling	FCFS	Tetris	DQN
Low task load	Avg response time	1.548s	1.311s	1.268s	0.981s	**0.888s**
	Success rate	89.8%	97.5%	98.1%	97.6%	**98.8%**
	Utilization rate	19.0%	18.9%	18.7%	12.6%	**13.1%**
Medium task load	Avg response time	6.291s	2.053s	1.339s	1.203s	**0.914s**
	Success rate	64.0%	84.9%	95.7%	92.6%	**98.0%**
	Utilization rate	47.5%	47.4%	46.3%	31.6%	**32.4%**
High task load	Avg response time	1232.369s	1177.950s	277.077s	3.043s	**1.034s**
	Success rate	31.6%	47.3%	53.3%	78.6%	**96.2%**
	Utilization rate	71.2%	71.1%	68.9%	47.4%	**48.9%**

same time period separately and categorize the task peaks into three types: low, medium, and high. The higher the task peak, the more obvious the trend of sudden increase of task load in a certain time period, and the higher the demand on the total processing power of the VM instances. Since the DRL agent needs time to learn how to make optimal decisions, we choose to start recording the results after the first day. The corresponding experimental results are shown in Tables 3, 4 and 5.

Table 3 shows the experimental results of different algorithms under different task loads. In the case of medium loads, for example, the random algorithm is the least effective in terms of average response time and success rate under the same load. The random algorithm and polling algorithm use system resources more than other algorithms and are not as effective under the same conditions. Their utilization rates are similar and higher than those of other algorithms. The DQN algorithm shows much better results than the other compared algorithms under different metrics, maintaining a low average response time and utilization while ensuring a high success rate. We compare the results under different task load scenarios, where the algorithms perform differently as the task load increases to complete the effect. In the case of Tetris, for example, as the task load increases, the average response time and utilization increase, and the success rate decreases, but the magnitude of the change is relatively small compared to the other compared algorithms. For the DQN algorithm, it shows better effects and magnitudes than all the other compared algorithms in the same load case.

Tables 4 and 5 show the experimental results under different task lengths and different task peaks, respectively. In the case of the same task length, the Tetirs algorithm is slightly better than the DQN algorithm in terms of utilization due to the characteristics of the Tetirs algorithm, but the DQN algorithm is far better in terms of other metrics. In the case of different task lengths, we can find that the difference in metrics between different algorithms is not significant when task lengths are short. However, as the length of the task increases and the demand of the task increases, the gap between the results of different algorithms becomes more prominent. The DQN algorithm shows more excellent results in different cases. At the same time, the DQN algorithm is able to maintain a certain degree of stability and robustness as the task situation changes. When there is a sudden increase in the task demand, the DQN algorithm performs well. It shows good response times, high success rates and efficient resource utilization, which means it can consistently handle sudden increases in tasks.

Table 4 Comparison results of scheduling algorithms with different task lengths

Categorization	Metric	Random	Polling	FCFS	Tetris	DQN
Short task length	Avg response time	0.823s	0.672s	0.652s	0.506s	**0.439s**
	Success rate	87.1%	97.5%	97.9%	97.5%	**98.9%**
	Utilization rate	23.7%	23.7%	23.4%	15.8%	**16.2%**
Medium task length	Avg response time	6.425s	2.177s	1.339s	1.203s	**0.923s**
	Success rate	64.3%	84.9%	95.8%	92.5%	**97.9%**
	Utilization rate	47.5%	47.5%	46.3%	31.5%	**32.2%**
Long task length	Avg response time	1175.790s	1168.215s	285.544s	6.878s	**1.540s**
	Success rate	31.7%	47.1%	53.1%	77.7%	**96.3%**
	Utilization rate	71.1%	71.2%	68.9%	47.5%	**48.7%**

Table 5 Comparison results of scheduling algorithms under different task peaks

Categorization	Metric	Random	Polling	FCFS	Tetris	DQN
Short task peak	Avg response time	1.530s	1.314s	1.273s	0.974s	**0.868s**
	Success rate	90.2%	97.5%	98.1%	97.7%	**98.9%**
	Utilization rate	18.7%	18.6%	18.5%	12.4%	**12.5%**
Medium task peak	Avg response time	7.206s	2.178s	1.318s	1.192s	**0.908s**
	Success rate	64.3%	85.1%	96.2%	92.6%	**98.3%**
	Utilization rate	47.5%	47.5%	46.2%	31.7%	**32.7%**
Long task peak	Avg response time	996.633s	947.137s	267.983s	4.663s	**1.028s**
	Success rate	32.3%	48.3%	51.7%	77.6%	**96.3%**
	Utilization rate	70.0%	70.0%	67.7%	46.6%	**48.0%**

5.5 Experiments with the dual DQN algorithm

In this subsection, experiments are conducted using simulated workloads and real-world workloads respectively. In order to evaluate the effectiveness of the dual DQN algorithm, we compare it with the single-layer DQN algorithm with fixed number of VM instances. Meanwhile, we propose two other double-layer comparison algorithms: FCFS-DQN algorithm and DQN-ARIMA algorithm. For FCFS-DQN algorithm, the first layer uses the FCFS algorithm for task scheduling and the second layer uses our designed DQN algorithm for resource adjustment. DQN-ARIMA algorithm uses the DQN algorithm for task scheduling and the AEIMA prediction algorithm for resource adjustment. At each autoscaling decision moment, i.e., every five minutes, the second layer of the dual scheduling approach will generate a VM adjustment decision to change the number of VM instances in the resource pool. The ARIMA algorithm collects the number of tasks executed in five minutes during the last ten resource adjustments and predicts the number of tasks in the next five minutes based on the collected data. For the predicted number of tasks, we use the average of the number of tasks in the ten resource adjustments as a benchmark and select 70% and 40% as the upper and lower thresholds, respectively. If the predicted value is greater than the upper threshold, one VM instance will be removed in the next phase, and if it is smaller than the lower threshold, it will decrease one VM instance in the next phase. The ARIMA model contains three parameters: the number of autoregressive terms p, the number of differences d, and the number of moving average terms q. The performance of the ARIMA model depends on these three parameters. If the number of autoregressive terms p is too small, it cannot capture the long-term dependence and is prone to underfitting. Conversely, if it is too large, it introduces noise and leads to overfitting. Too small a number of differences d will lead to sequence instability and thus model failure, and too large a number will lead to loss of valid information. If the number of moving average terms q is too small, the model will ignore the error structure, leading to large predicted errors. On the other hand, if it is too large, it complicate the model and increase the computation time. After several experiments, it was finally determined that p=1, d=1, and q=1. We first apply the above algorithms to handle the similar workloads mentioned in Section 5.4.

Figure 5 shows the experimental results of our dual DQN algorithm in the simulated environment for eight days. The x-axis represents the number of days, and the y-axis represents the change in the number of tasks and the number of VM units every five minutes, respectively. The figure depicts the variation in the number of rented VM instances with task workloads using the dual DQN algorithm. As shown in the figure, the first three days of learning typically involve 8 to 10 VM instances. During the fourth and fifth days of the learning process, the number of VM instances starts to gradually adapt to the workload trend, and the change in the number of VM instances starts to fit the change in the number of tasks. As for the last three days, the change of the number of VM instances has completely adapted to the change of the workloads. During low workload periods, the number of VM instances tends to be 3 to 5, while during high workload periods, the number of VM instances tends to be 8 to 10.

Table 6 displays the specific experimental results of all algorithms in the simulated environment. It can be found that the dual DQN algorithm has a higher success rate and lower renting costs compared to the single-layer DQN algorithm with fixed 10 VM instances, even though the average response time for the dual DQN is a bit longer. It reduces the rental cost by approximately 43%. Compared to the single-layer DQN algorithm with fixed 3 VM instances, the dual DQN algorithm significantly outperforms the DQN-3VM algorithm in terms of average response time and success rate, and it also exhibits advantages over the DQN-3VM

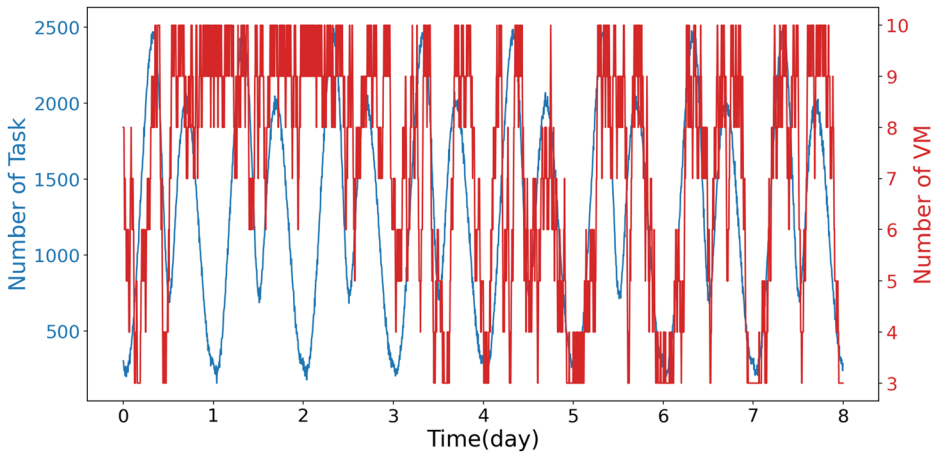

Fig. 5 Variation of workload and the number of VM instances using the dual scheduling approach

algorithm in terms of rental cost. In the comparison of the double-layer algorithms, we use the DQN algorithm in the first and second layers respectively. As shown in Table 6, the dual DQN algorithm not only improves the average response time and success rate, but also saves 24.8% and 21.1% in rental costs compared to the FCFS-DQN algorithm and the DQN-ARIMA algorithm respectively.

To further validate the performance of the dual DQN algorithm, we used a real NASA dataset (Arlitt and Williamson 1996) to test. The NASA workload trace we used contains all HTTP requests made during July 1995 by NASA's WWW server at the Kennedy Space Center in Florida. We recorded experimental results of the Dual DQN algorithm and other comparison algorithms for the whole month in Table 7.

As can be seen in Table 7, in terms of the task average response time and success rate, the performance of the single-layer DQN algorithm with fixed 10 VM instances and the dual DQN algorithm are much better than the other compared algorithms. However, the single-layer DQN algorithm cannot dynamically adjust the number of leased VM instances according to workload changes, which leads to wasted resources and increased leasing costs. The dual DQN algorithm can balance the optimization objectives of both QoS and cost. Although the average response time and success rate of the dual DQN algorithm is slightly worse than that of the single-layer DQN algorithm with fixed 10 VM instances, the cost savings is approximately 43.3%.

Due to the large number of tasks in a month, Figure 6 only shows the workload and the variation in the number of leased VM instances for the first 8 days using the dual DQN algorithm. Similar to the learning process shown in Fig. 5, the change in the number of rented VM instances also gradually adapts to the workload trend.

When comparing the two cases in Figs. 5 and 6, it shows that in the simulated environment, the Dual DQN algorithm improved from its initial strategy to a nearly optimal one within five days. This is because, even though the number of tasks varied slightly at different times, the overall trend remained stable, allowing for faster improvement. However, in the real-world environment, the overall trend and daily data varied significantly, As a result, the algorithm only managed to reach a reasonable level in the last two days, which means it took longer to improve compared with the simulated environment. In the real-world scenario due to the large variability in the overall trend and the daily data volume, only the last two

Table 6 Comparison results for the dual DQN Algorithm

	Avg response time	Success rate	Rental cost
DQN-10VM	0.958s	99.7%	7205
DQN-3VM	256.39s	68.1%	5039
FCFS-DQN	7.58s	85.1%	5432
DQN-ARIMA	5.88s	90.5%	5178
Dual DQN	**1.01s**	**99.6%**	**4084**

Table 7 Comparison results for the dual DQN Algorithm in real environment

	Avg response time	Success rate	Rental cost
DQN-10VM	437ms	96.9%	43502.5
DQN-2VM	899ms	87.8%	8700.5
FCFS-DQN	786ms	74.1%	30070.9
DQN-ARIMA	610ms	94.7%	26193.4
Dual DQN	**472ms**	**95.8%**	**20271.1**

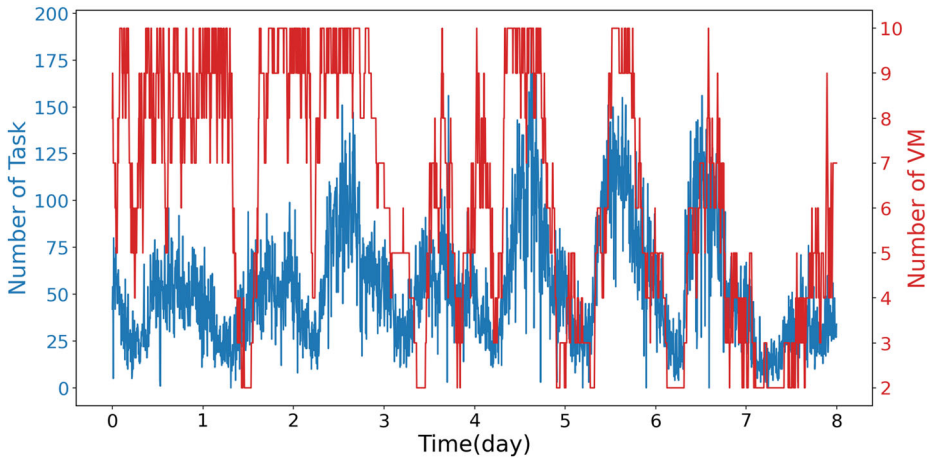

Fig. 6 Changes in workload and the number of VM instances using the dual scheduling approach in the real scenario

days can converge to a reasonable range, which represents a certain decrease in convergence speed compared with the simulated environment. For the convergence stability, due to the similarity of the simulated environment, the Dual DQN algorithm undergoes three days of trial and error followed by two days of learning and updating and finally converges to a stable range, which adjusting the number of resources according to the change in the number of tasks changes. In the real-world environment, because of its unpredictability, the Dual DQN algorithm shows more fluctuation during both the trial and error phase and the learning and updating phase compared with the simulated environment, but it eventually settles into a stable range based on the number of tasks.

6 Conclusion and future work

Efficient task scheduling and elastic resource provisioning are two major challenges faced by application providers. In this paper, we address this issue and propose an intelligent dual scheduling framework. In this framework, we use a dual scheduling structure based on DRL, where the first layer DQN algorithm is responsible for assigning tasks to appropriate VM instances, and the second layer DQN algorithm is responsible for dynamically adjusting the number of VM instances to reduce the cost of renting VM instances. We compare the single-layer DQN algorithm and the dual DQN algorithm with other baseline algorithms, respectively. Experimental results show that our approach is effective in ensuring high success rates and significantly reducing the rental cost. In the future, we plan to extend the dual scheduling framework to achieve hybrid task scheduling in a more realistic and complex cloud environment. For example, handling independent tasks and workflow tasks simultaneously, considering different VM pricing models, and other factors.

Acknowledgements This work was supported by the Natural Science Foundation of Shandong Province, China (Grant number ZR2020QF026).

Author Contributions Methodology: Jiahui Pan, Yi Wei, and Lei Meng; Conceptualization: Jiahui Pan and Yi Wei; Visualization: Jiahui Pan; Writing-Original Draft & Review & Editing: Jiahui Pan; Investigation: Yi Wei; Writing-Review & Editing: Yi Wei, Lei Meng, and Xiangxu Meng; Supervision: Yi Wei, Lei Meng, and Xiangxu Meng.

Data Availability Data will be made available on request.

Declarations

Competing Interests The authors have no relevant financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Abid A, Manzoor MF, Farooq MS et al (2020) Challenges and issues of resource allocation techniques in cloud computing. KSII Trans Internet Inf Syst (TIIS) 14(7):2815–2839. https://doi.org/10.3837/tiis.2020.07.005

Alhaidari F, Balharith TZ (2021) Enhanced round-robin algorithm in the cloud computing environment for optimal task scheduling. Computers 10(5):63. https://doi.org/10.3390/computers10050063

Alharthi S, Alshamsi A, Alseiari A et al (2024) Auto-scaling techniques in cloud computing: Issues and research directions. Sensors 24(17):5551. https://doi.org/10.3390/s24175551

Arlitt MF, Williamson CL (1996) Web server workload characterization: The search for invariants. ACM SIGMETRICS Performance Evaluation Review 24(1):126–137. Traces available at https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html

Arunarani A, Manjula D, Sugumaran V (2019) Task scheduling techniques in cloud computing: A literature survey. Futur Gener Comput Syst 91:407–415. https://doi.org/10.1016/j.future.2018.09.014

Barnawi A, Sakr S, Xiao W et al (2020) The views, measurements and challenges of elasticity in the cloud: A review. Comput Commun 154:111–117. https://doi.org/10.1016/j.comcom.2020.02.010

Belgacem A, Mahmoudi S, Kihl M (2022) Intelligent multi-agent reinforcement learning model for resources allocation in cloud computing. J King Saud University-Comput Inform Sci 34(6):2391–2404

Cheng Y, Cao Z, Zhang X et al (2024) Multi objective dynamic task scheduling optimization algorithm based on deep reinforcement learning. J Supercomput 80(5):6917–6945

Clifton J, Laber E (2020) Q-learning: Theory and applications. Annual Review Stat Appl 7(1):279–301. https://doi.org/10.1146/annurev-statistics-031219-041220

Ding Z, Huang Y, Yuan H, et al (2020) Introduction to reinforcement learning. Deep reinforcement learning: fundamentals, research and applications pp 47–123. https://doi.org/10.1007/978-981-15-4095-0_2

Garí Y, Monge DA, Pacini E et al (2021) Reinforcement learning-based application autoscaling in the cloud: A survey. Eng Appl Artif Intell 102:104288. https://doi.org/10.1016/j.engappai.2021.104288

Garí Y, Monge DA, Mateos C (2022) A q-learning approach for the autoscaling of scientific workflows in the cloud. Futur Gener Comput Syst 127:168–180. https://doi.org/10.1016/j.future.2021.09.007

Grandl R, Ananthanarayanan G, Kandula S et al (2014) Multi-resource packing for cluster schedulers. ACM SIGCOMM Comput Commun Rev 44(4):455–466. https://doi.org/10.1145/2740070.2626334

Hegde A, Kulkarni SG, Prasad AS (2023) Counsel: Cloud resource configuration management using deep reinforcement learning. 2023 IEEE/ACM 23rd International Symposium on Cluster. Cloud and internet computing (CCGrid), IEEE, pp 286–298

Hou H, Jawaddi SNA, Ismail A (2024) Energy efficient task scheduling based on deep reinforcement learning in cloud environment: A specialized review. Futur Gener Comput Syst 151:214–231. https://doi.org/10.1016/j.future.2023.10.002

Huang Q, Liu H, Zhai X (2023) Dsn: A ddpg-based scheduling framework for optimal task allocation in cloud data centers. 2023 2nd International Conference on Cloud Computing. Big Data Application and Software Engineering (CBASE), IEEE, pp 324–329

Ibrahim IM, Zeebaree SR, et al (2021) Task scheduling algorithms in cloud computing: A review. Turkish J Comput Math Educ (TURCOMAT) 12(4):1041–1053. https://doi.org/10.17762/turcomat.v12i4.612

Kakkottakath JValappil Thekkepuryil, Suseelan DP, Keerikkattil PM (2021) An effective meta-heuristic based multi-objective hybrid optimization method for workflow scheduling in cloud computing environment. Clust Comput 24(3):2367–2384. https://doi.org/10.1016/j.future.2017.03.024

Karthiban K, Raj JS (2020) An efficient green computing fair resource allocation in cloud computing using modified deep reinforcement learning algorithm. Soft Computing-A Fusion of Foundations, Methodologies & Applications 24(19). https://doi.org/10.1007/s00500-020-04846-3

Kaur R, Laxmi V, Balkrishan (2022) Performance evaluation of task scheduling algorithms in virtual cloud environment to minimize makespan. International Journal of Information Technology pp 1–15. https://doi.org/10.1007/s41870-021-00753-4

Khan H, Usman MT, Rida I et al (2024) Attention enhanced machine instinctive vision with human-inspired saliency detection. Image and Vision Computing 152:105308

Khan H, Usman MT, Koo J (2025) Bilateral feature fusion with hexagonal attention for robust saliency detection under uncertain environments. Information Fusion p 103165

Khan T, Tian W, Zhou G et al (2022) Machine learning (ml)-centric resource management in cloud computing: A review and future directions. Journal of Network and Computer Applications 204:103405. https://doi.org/10.1016/j.jnca.2022.103405

Kruekaew B, Kimpan W (2022) Multi-objective task scheduling optimization for load balancing in cloud computing environment using hybrid artificial bee colony algorithm with reinforcement learning. IEEE Access 10:17803–17818. https://doi.org/10.1109/ACCESS.2022.3149955

Kumar M, Sharma SC (2018) Deadline constrained based dynamic load balancing algorithm with elasticity in cloud environment. Comput Electr Eng 69:395–411. https://doi.org/10.1016/j.compeleceng.2017.11.018

Lee I (2021) Pricing and profit management models for saas providers and iaas providers. J Theor Appl Electron Commer Res 16(4):859–873. https://doi.org/10.3390/jtaer16040049

Li M, Pei P, Yu FR et al (2022) Cloud-edge collaborative resource allocation for blockchain-enabled internet of things: A collective reinforcement learning approach. IEEE Internet Things J 9(22):23115–23129

Liu B, Guo J, Li C et al (2020) Workload forecasting based elastic resource management in edge cloud. Comput Ind Eng 139:106136. https://doi.org/10.1016/j.cie.2019.106136

Mao H, Alizadeh M, Menache I, et al (2016) Resource management with deep reinforcement learning. In: Proceedings of the 15th ACM workshop on hot topics in networks, pp 50–56

Mao H, Schwarzkopf M, Venkatakrishnan SB, et al (2019) Learning scheduling algorithms for data processing clusters. In: Proceedings of the ACM special interest group on data communication. p 270–288

Meng Z, Wang M, Bai J, et al (2020) Interpreting deep learning-based networking systems. In: Proceedings of the annual conference of the ACM special interest group on data communication on the applications, technologies, architectures, and protocols for computer communication, pp 154–171

Mohammed CM, Zeebaree SR (2021) Sufficient comparison among cloud computing services: Iaas, paas, and saas: A review. International Journal of Science and Business 5(2):17–30. URL https://ideas.repec.org/a/aif/journl/v5y2021i2p17-30.html

Nabi S, Ahmad M, Ibrahim M et al (2022) Adpso: adaptive pso-based task scheduling approach for cloud computing. Sensors 22(3):920. https://doi.org/10.3390/s22030920

Polamarasetti A (2024) Machine learning techniques analysis to efficient resource provisioning for elastic cloud services. In: 2024 International Conference on Intelligent Computing and Emerging Communication Technologies (ICEC), IEEE, pp 1–6

Pradhan A, Bisoy SK, Das A (2022) A survey on pso based meta-heuristic scheduling mechanism in cloud computing environment. Journal of King Saud University-Computer and Information Sciences 34(8):4888–4901. https://doi.org/10.1016/j.jksuci.2021.01.003

Prajapati Y, Gosai K, Suthar O, et al (2025) Privacy and security concerns with 6g smart city infrastructure. In: Building Tomorrow's Smart Cities With 6G Infrastructure Technology. IGI Global Scientific Publishing, p 113–136

Psychas K, Ghaderi J (2018) Randomized algorithms for scheduling multi-resource jobs in the cloud. IEEE/ACM Transactions on Networking 26(5):2202–2215. https://doi.org/10.1109/TNET.2018.2863647

Rashid A, Chaturvedi A (2019) Cloud computing characteristics and services: a brief review. International Journal of Computer Sciences and Engineering 7(2):421–426. https://doi.org/10.26438/ijcse/v7i2.421426

Rzadca K, Findeisen P, Swiderski J, et al (2020) Autopilot: workload autoscaling at google. In: Proceedings of the Fifteenth European Conference on Computer Systems, pp 1–16, https://doi.org/10.1145/3342195.3387524

Samek W, Montavon G, Lapuschkin S et al (2021) Explaining deep neural networks and beyond: A review of methods and applications. Proceedings of the IEEE 109(3):247–278. https://doi.org/10.1109/JPROC.2021.3060483

Shyalika C, Silva T, Karunananda A (2020) Reinforcement learning in dynamic task scheduling: A review. SN Computer Science 1(6):306. https://doi.org/10.1007/s42979-020-00326-5

Singh P, Gupta P, Jyoti K, et al (2019) Research on auto-scaling of web applications in cloud: survey, trends and future directions. Scalable Computing: Practice and Experience 20(2):399–432. https://doi.org/10.12694/scpe.v20i2.1537

Sutton RS (2018) Reinforcement learning: An introduction. A Bradford Book URL http://creativecommons.org/licenses/by-nc-nd/2.0/

Syed I (2020) Hamm: A hybrid algorithm of min-min and max-min task scheduling algorithms in cloud computing. International Journal of Recent Technology and Engineering (IJRTE) 9:209–218. URL https://ssrn.com/abstract=3922243

Tournaire T, Castel-Taleb H, Hyon E (2023) Efficient computation of optimal thresholds in cloud auto-scaling systems. ACM Transactions on Modeling and Performance Evaluation of Computing Systems 8(4):1–31. https://doi.org/10.1145/3603532

Ullah I, Singh SK, Adhikari D et al (2025) Multi-agent reinforcement learning for task allocation in the internet of vehicles: Exploring benefits and paving the future. Swarm and Evolutionary Computation 94:101878

Wang B, Wang C, Song Y et al (2020) A survey and taxonomy on workload scheduling and resource provisioning in hybrid clouds. Cluster Computing 23:2809–2834. https://doi.org/10.1007/s10586-020-03048-8

Wang X, Wang S, Liang X et al (2022) Deep reinforcement learning: A survey. IEEE Transactions on Neural Networks and Learning Systems 35(4):5064–5078. https://doi.org/10.1109/TNNLS.2022.3207346

Wang Y, Yang X (2025) Research on edge computing and cloud collaborative resource scheduling optimization based on deep reinforcement learning. arXiv:2502.18773

Wei Y, Pan L, Liu S et al (2018) Drl-scheduling: An intelligent qos-aware job scheduling framework for applications in clouds. IEEE access 6:55112–55125. https://doi.org/10.1109/ACCESS.2018.2872674

Wei Y, Kudenko D, Liu S, et al (2019) A reinforcement learning based auto-scaling approach for saas providers in dynamic cloud environment. Mathematical Problems in Engineering 2019(1):5080647. https://doi.org/10.1155/2019/5080647

Yan J, Huang Y, Gupta A et al (2022) Energy-aware systems for real-time job scheduling in cloud data centers: A deep reinforcement learning approach. Computers and Electrical Engineering 99:107688. https://doi.org/10.1016/j.compeleceng.2022.107688

Yang J, Jiang B, Lv Z et al (2020) A task scheduling algorithm considering game theory designed for energy management in cloud computing. Future Generation computer systems 105:985–992. https://doi.org/10.1016/j.future.2017.03.024

Zou W, Zhang Z, Wang N et al (2025) Reflexpilot: Startup-aware dependent task scheduling based on deep reinforcement learning for edge-cloud collaborative computing. IEEE Transactions on Cloud Computing

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.